

What Can Multi-LLM Agents Actually Do for Software Security?

An Empirical Study of Vulnerability Detection and *Executable-Test-Verified* Repair

Hinimdou Morsia Guitdam¹ Djeri-Alassani Oubenoupu¹ Chaïbou Saidou Abdoulaye¹

¹École Nationale Supérieure d'Arts et Métiers de Meknès (ENSAM)

AI & Intelligent Systems Laboratory

hinimdoumorsia@gmail.com djeryala@gmail.com a.chaibou.tech@gmail.com

Supervisor: Prof. Hajji Tarik¹
June 1, 2026

Keywords: Empirical study, Vulnerability detection, Automated program repair, Proof-of-Vulnerability tests, Multi-agent system, LLM, Ablation study, SAST, OWASP, CVEfixes, Vul4J.

Abstract

LLM-based coding agents are routinely evaluated on functional bug resolution, leaving two questions under-measured: do patches *preserve* security, and does repair capability survive *executable test* verification (vs. textual similarity)?

We present a controlled empirical study using MultiAgentSecurite, an orchestration (LangGraph) of existing SAST tools, a Rust memory-safety engine, and LLM reasoning, evaluated via formal causal ablation across four benchmarks: OWASP Benchmark ($n = 2740$), Juliet NIST ($n = 200$), CVEfixes ($n = 798$), and Vul4J ($n = 183$ evaluable cases, PoV executable tests as gold standard). All experiments use fixed seeds, frozen model IDs, and Wilson/bootstrap confidence intervals.

Key findings: (1) multi-SAST orchestration raises OWASP recall $0.79 \rightarrow 0.96$ (disjoint CIs), supporting H1; (2) Rust engine underperforms LLM on memory CWEs—recall 0.19 vs. 1.00 (**directional**, $n = 21$, **primary limitation**, H2 refuted directionally); (3) detection/repair decoupling confirmed across 10 models ($\rho_s = -0.71$, $p = 0.021$): no model simultaneously achieves Youden $J > 0$ and fix-rate $> 10\%$; (4) PoV-verified repair rate 12.3% (CI [10.4; 14.4]%), vs. 8.7% direct LLM call (+15.9 absolute pts, McNemar $p < 0.001$); (5) no new vulnerability detected by bi-scanner over 183 patches (syntactic lower bound, not a security guarantee). The RAG component is deferred to future work.

1. INTRODUCTION

1.1 Context and Motivation

Integrating LLM-based agents into CI/CD pipelines raises fundamental questions of reliability and security. Code completion tools such as GitHub Copilot [26] are now widespread, and autonomous agents resolve entire GitHub issues (Devin [25], SWE-agent [21]). Recent studies show that 40% of Copilot suggestions on security-related tasks contain vulnerabilities [13], and that LLMs introduce new flaws in 20% of their bug fixes [16]. Yet these systems are almost exclusively

measured on *functional resolution* (a unit test turns green) — not on whether the modification *preserves* the project’s security posture. This gap between *what is optimized* and *what is measured* is the starting point of our study.

1.2 Research Questions

This work *empirically interrogates* rather than claims what state-of-the-art multi-agent systems can and cannot do. Three operational and falsifiable research questions structure the study:

RQ1 (Detection): Does combining multiple static analyses (SAST), a specialized memory engine, and LLM reasoning *measurably* improve recall, and at what false-positive cost?

RQ2 (Specialization): On memory corruptions, does a specialized rule engine outperform a general-purpose LLM, as commonly assumed?

RQ3 (Repair & Safety): Does automated repair hold when judged by *executable tests* rather than similarity, and can one *measure* that it introduces no new flaw?

Each RQ is associated with a *refutable* hypothesis (H1–H3, Section 4.1); refutations are reported on equal footing with confirmations.

The related work is surveyed in Section 2.

1.3 Contributions

1. **A reproducible evaluation protocol** for multi-LLM security agents: fixed seeds, frozen model IDs, bootstrap and Wilson CIs, PoV test verification. The protocol is *reusable* with any LLM set or SAST toolchain.
2. **Five empirical findings across four benchmarks** ($n = 183$ evaluable Vul4J cases): *Established with disjoint CIs*: multi-SAST synergy (OWASP R: $0.79 \rightarrow 0.96$); H2 refuted directionally ($n = 21$, preliminary); LLM decisive on real code, null on synthetic; similarity overestimates repair capability. *Now confirmed at $\alpha = 0.05$* : detection/repair decoupling ($\rho_s = -0.71$, $p = 0.021$, $n = 10$ models; no model achieves $J > 0$ AND fix-rate $> 10\%$). The orchestrated pipeline outperforms a direct LLM call by +15.9 absolute points (24.6% vs. 8.7%) on Vul4J PoV fix-rate.
3. **A controlled ablation study** isolating each component’s contribution via unit flag disabling, all other parameters held constant.
4. **An external baseline comparison** (CodeQL v2.25.6) under identical matching, objectively situating the recall/precision trade-off.
5. **A bi-scanner patch safety measurement** (Semgrep + SpotBugs, 183 patches), bounding

syntactically-detectable secondary vulnerability introduction.

6. A reusable evaluation framework (MultiAgentSecurite, code and data on GitHub) instantiable with new LLMs or tools.

1.4 Scope Delimitation

This work proposes no new detection or repair algorithm: the architecture reuses existing tools (Semgrep, SpotBugs, LLM, LangGraph) and constitutes the *measurement instrument*, not a novelty claim. Novelty is *methodological and empirical*: a PoV-verified protocol, a formalized ablation study, and reproducible findings including negative results. The patch safety measure (H3) is bounded but not exhaustive, as discussed in Section 6.6.

2. RELATED WORK

2.1 Automated Program Repair (APR)

The field has progressed from evolutionary (GenProg [7]), statistical (Prophet [8]), and template-based approaches (TBar [10]: 43 % on Defects4J [2]). Recent surveys by Monperrus [9], Ghaleb *et al.* [22], and Mariano *et al.* [24] document the state of the art. *Critical gap common to all APR approaches*: functional correction is optimized without security preservation criteria. None of these works measures whether a patch preserves the security posture or introduces a new vulnerability. This gap directly motivates RQ3.

2.2 LLM-based Vulnerability Detection and Repair

Pearce *et al.* [13] evaluate zero-shot C/C++ CVE repair (success drops from 67 % on simple to 23 % on complex cases). *Gaps*: single model, no ablation, no executable test verification, no security regression measurement.

VulRepair [14] (T5 fine-tuned on CVEfixes) reports 47 % repair. *Critical gaps*: (a) measurement by BLEU/CodeBLEU, not executable tests — Section 6.2 demonstrates that similarity systematically overestimates real repair; (b) CVEfixes “negatives” are post-fix versions that may still be vulnerable, rendering recall/precision unreliable; (c) no causal ablation; (d) no secondary vulnerability measurement.

Structural GNN approaches (Devign [17], ReVeal [18], LineVul [19]) are detection-only. *Specific gaps*: Devign propagates project-level vulnerability labels to individual functions, introducing label noise that inflates reported recall; LineVul’s line-level localization is evaluated without ablation to separate model architecture from training-data effects; neither defines formally what constitutes a true positive under ambiguous code semantics. Recent code LLMs (CodeLlama [35], StarCoder2 [36]) provide strong code completion but are evaluated on functional correctness (HumanEval, MBPP), not on security-specific metrics with executable tests.

2.3 Multi-Agent Systems for Code

AgentCoder [20] (+15 % on HumanEval) and SWE-agent [21] (30.3 % on SWE-bench) demonstrate that

LLM agents can resolve functional issues. *Gaps*: no vulnerability re-scan after repair, no PoV test verification, no formal component ablation. These works confirm that an LLM can *resolve* a reported issue, but not that it does so without degrading the security posture.

2.4 Security Benchmarks and Positioning

Defects4J [2] is the standard APR corpus; OWASP Benchmark [3] ($n = 2740$ Java cases, clean negatives) serves SAST precision evaluation; Juliet [4] (NIST, 200 C/C++ cases) targets memory-safety CWEs; CVEfixes [5] (798 multi-language real CVEs, noisy negatives) provides ecological validity; Vul4J [6] (Java vulnerabilities with *executable* PoV tests) is the only benchmark enabling hard repair verification — our gold standard.

Recent benchmarks complement these. SecurityEval [33] (75 CWE scenarios) measures *secure code generation* — a different task from detection/repair. CyberSecEval [34] (8-dimension meta-benchmark) includes insecure-code detection; its finding that models specializing in one security task underperform on others directly parallels our Finding 3 ($\rho_s = -0.71$). Neither benchmark provides PoV executable tests, which is the core methodological distinction of this work.

Table 1 situates our approach relative to prior work.

Table 1. Positioning vs. related work. PoV = Proof-of-Vulnerability tests; Sec. = security preservation measured; Abl. = ablation study. ✓ = yes; × = no; ◦ = functional tests only.

Study	Multi-SAST	LLM	PoV	Sec.+Abl.
VulRepair [14]	×	✓	×	×
SWE-agent [21]	×	✓	◦	×
Pearce <i>et al.</i> [13]	×	✓	×	partial
Devign [17]	×	×	×	×
AgentCoder [20]	×	✓	◦	×
This work	✓	✓	✓	✓

2.5 Scientific Gap and Positioning

A **double blind spot** persists in the literature: (a) *security preservation* by automated patches is rarely quantified; (b) *repair* is almost always reported via textual similarity, a proxy we demonstrate overestimates real capability by more than 3×. Our contribution is a rigorous, reproducible, multi-faceted measurement of what state-of-the-art components, assembled and ablated in a controlled fashion, succeed and fail at on both fronts. To our knowledge, no prior study combines formal ablation, multi-SAST orchestration, and PoV-verified repair in a single reproducible protocol while simultaneously quantifying secondary vulnerability introduction.

3. ARCHITECTURE OF THE MEASUREMENT INSTRUMENT

The architecture constitutes the study’s *instrument*: it assembles state-of-the-art components to *isolate and compare* them via ablation. Eight specialized agents are coordinated via **LangGraph** [31] around a shared state

(AgentState), ensuring traceability and reproducibility. Figure 1 gives the overview.

3.1 Indexing Strategy and Triage

Unlike *long-context* approaches that saturate the LLM with the full codebase, the **TriageAgent** builds a lightweight dependency graph. It detects present languages (C, C++, Python, JavaScript, TypeScript, Rust, Java, Go, PHP) and extracts only affected files and their direct dependencies, optimizing the context window and selectively triggering appropriate tools.

3.2 Hybrid Analysis: SAST and LLM

The system combines three complementary layers whose contributions we measure separately:

1. **Static (ScannerAgent)**: parallel orchestration of five SAST engines — Sengrep [27] (multi-language), Bandit (Python), Gosec (Go), SpotBugs+FindSecBugs [28] (Java), PHPCS (PHP) — via `ThreadPoolExecutor` with 4 workers and SHA-256 cache (Figure 2).
2. **Memory (MemorySafetyAgent)**: independent Rust engine, rules MEM-001 to MEM-081. Section 5.3 shows this engine, while precise, has very low recall: it serves as a *comparison term* in the H2 ablation, not as a performance argument.
3. **Semantic (SemanticAnalystAgent)**: LLM Llama-3.1-8B [29] (Groq, free-tier), designed to be enriched by RAG. Memory was not populated at evaluation time; all results are zero-shot in the main evaluation (Section 3.7).

3.3 Agent Summary Table

Table 2. Pipeline agents and their execution properties.

#	Agent	Role	Mode
1	TriageAgent	Language and target file detection	Sequential
2	ScannerAgent	SAST static analysis (5 tools)	Sequential
3	MemorySafetyAgent	Memory safety (Rust, MEM rules)	Parallel
4	SemanticAnalyst	Logic flaws (LLM + RAG)	Parallel
5	ExploitScorer	CVSS 3.1 score, P1-P3 priority	Sequential
6	PatcherAgent	Patch generation (full file)	Sequential
7	ValidatorAgent	Patch and regression validation	Sequential
8	ReportAgent	Final JSON / Markdown report	Sequential

3.4 Execution Workflow and Conditional Routing

The workflow (Figure 3) is driven by five routing functions. `route_after_triage` routes to `ScannerAgent` or `ReportAgent` based on errors. `route_after_exploit_scorer` filters exploitable cases ($CVSS \geq 7.0$) before `PatcherAgent`. `route_after_validator` implements the retry loop (at most 3 iterations). All paths converge to `ReportAgent` (Algorithm 3).

Table 3. MultiAgentSecurite Detection Loop

- 1: **Require:** Repository $\mathcal{R}$, configuration $\theta = (T, CVSS_{\min}, N_{\max})$
- 2: **Ensure:** Vulnerability report $\mathcal{V}$, validated patches $\mathcal{P}$
- 3: $langs, targets \leftarrow TRIAGEAGENT(\mathcal{R})$
- 4: **if** error in targets **then**
- 5: **return** `REPORTAGENT`($\emptyset$)
- 6: **end if**
- 7: $\mathcal{V}_{sast} \leftarrow SCANNERAGENT(targets)$ // parallel SAST
- 8: $\mathcal{V}_{mem} \leftarrow MEMORYSAFETYAGENT(targets)$
- 9: $\mathcal{V}_{sem} \leftarrow SEMANTICANALYSTAGENT(targets)$
- 10: $\mathcal{V} \leftarrow \mathcal{V}_{sast} \cup \mathcal{V}_{mem} \cup \mathcal{V}_{sem}$
- 11: $\mathcal{V}^+ \leftarrow \{v \in \mathcal{V} \mid CVSS(v) \geq CVSS_{\min}\}$
- 12: **if** $\mathcal{V}^+ = \emptyset$ **then**
- 13: **return** `REPORTAGENT`($\mathcal{V}$)
- 14: **end if**
- 15: $\mathcal{P} \leftarrow PATCHREPAIRLOOP(\mathcal{V}^+, N_{\max})$
- 16: **return** `REPORTAGENT`($\mathcal{V}, \mathcal{P}$)

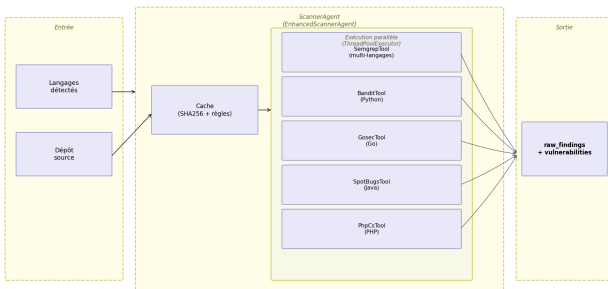


Figure 2. Parallel SAST tool orchestration. SHA-256 cache avoids rescans; the five tools run simultaneously via `ThreadPoolExecutor`.

3.5 Shared State: AgentState

AgentState (Figure 4) is the central data contract traversing the LangGraph workflow, ensuring traceability and avoiding side effects. It aggregates: (i) **configuration** (`repo_root`, `scan_id` UUID, detected languages, `max_patch_iterations=3`); (ii) **targets** (path, content truncated at 8000 chars, SHA-256 hash); (iii) **three separate vulnerability lists by source** — SAST, Rust memory, and LLM semantic — enabling per-component evaluation in Section 5.5; (iv) **patch queues** and iteration counter bounding the retry loop.

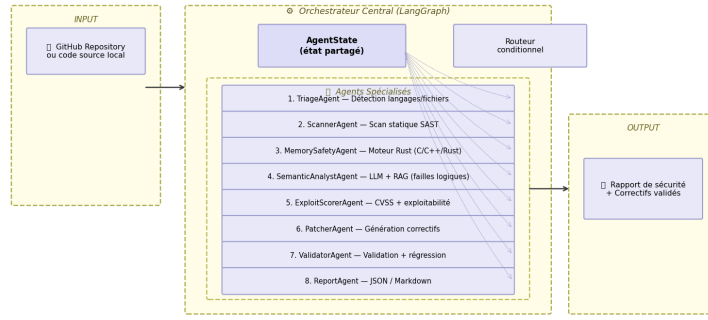


Figure 1. MultiAgentSecurite general architecture. The LangGraph orchestrator manages a shared **AgentState** distributed among eight specialized agents in a sequential flow with conditional branching.

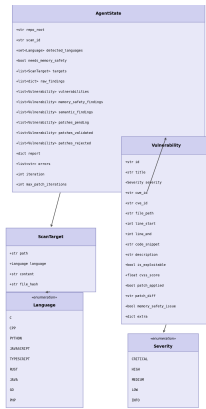
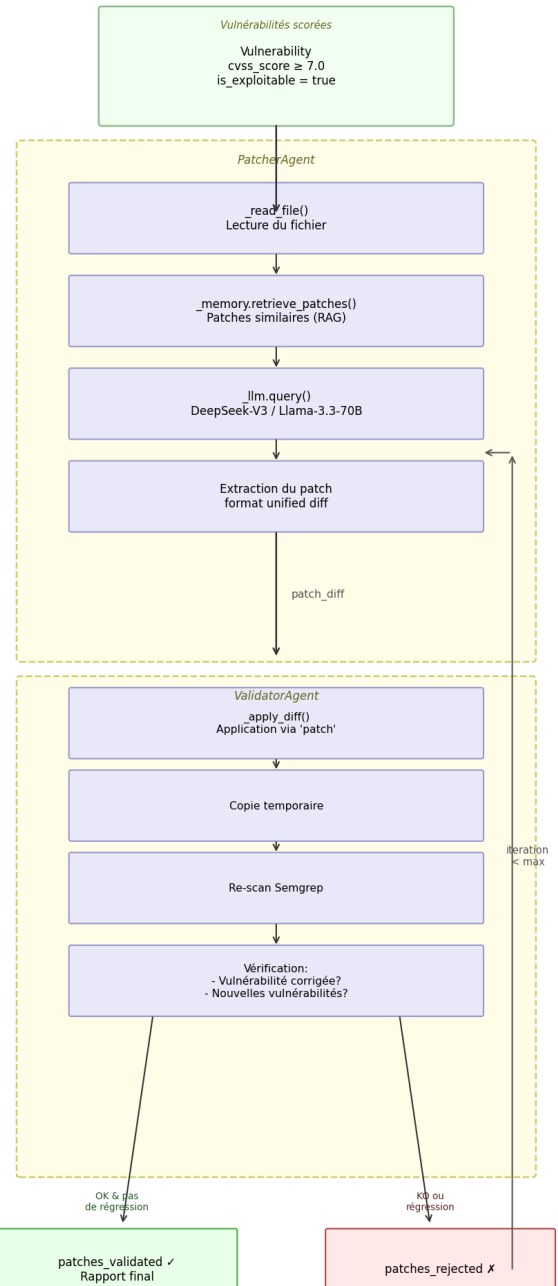


Figure 4. UML diagram of **AgentState** with **ScanTarget**, **Vulnerability**, and enumerations **Language** and **Severity**.



3.6 Automated Repair Pipeline

Figure 5 details the PatcherAgent–ValidatorAgent pipeline (Algorithm 4). PatcherAgent generates the corrected full file via DeepSeek-V3 [30]; the diff is re-computed locally by `difflib`. ValidatorAgent accepts a patch only if (i) the original vulnerability has disappeared *and* (ii) no new flaw is detected, otherwise retries up to 3 times.

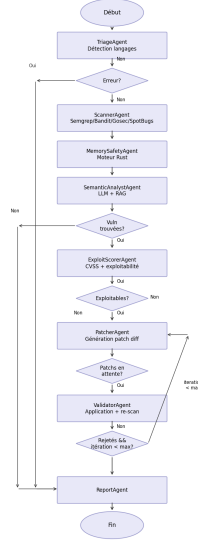


Figure 3. Execution workflow with conditional routing and retry loop.

Table 4. PatcherAgent–ValidatorAgent Repair Loop

```

1: Require:  $V$  vulnerability  $v$ ,  $N_{\max} = 3$ , LLM  $\mathcal{M}$ 
2: Ensure: Patch  $p^*$  or REJECTED
3:  $n \leftarrow 0$ 
4: while  $n < N_{\max}$  do
5:    $c \leftarrow \text{READFILE}(v.\text{file\_path})$ 
6:    $r \leftarrow \text{RAGRETRIEVE}(v.\text{cwe\_id})$ 
7:    $c' \leftarrow \mathcal{M}.\text{QUERY}(c, v, r)$  // generate patched file
8:    $p \leftarrow \text{DIFF}(c, c')$ 
9:    $\mathcal{V}' \leftarrow \text{RESCAN}(c')$  // SAST on patched file
10:  if  $v \notin \mathcal{V}'$  and  $|\mathcal{V}' \setminus \mathcal{V}| = 0$  then
11:    return  $p$  // vuln fixed, no regression
12:  end if
13:   $n \leftarrow n + 1$ 
14: end while
15: return REJECTED
    
```

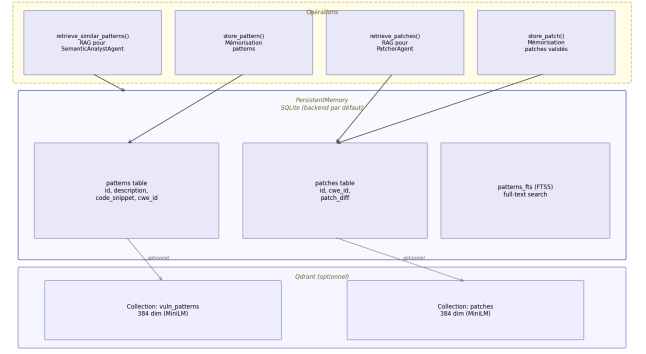


Figure 6. Persistent memory system (SQLite / Qdrant) — not activated in main evaluation runs.

3.7 Persistent Memory and RAG Pilot Evaluation

The RAG system (Figure 6) provides two backends: SQLite (default, `patterns/patches` tables, FTS5 index) and Qdrant (vectors dim. 384, *all-MiniLM-L6-v2*). The main evaluation is conducted in *zero-shot* (memory unpopulated) to isolate other components’ contributions. A **separate RAG pilot evaluation** on 60 CVEfixes cases (seed=42, memory populated with 200 training pairs) yields recall $0.24 \rightarrow 0.31$ with partially overlapping CIs — a positive trend reported in Section 7.4.

4. METHODOLOGY, FORMALISM, AND HYPOTHESES

4.1 Testable Hypotheses

- **H1 (Multi-engine synergy — RQ1):** SAST orchestration maximizes recall ($R > 0.90$) vs. any single tool. *Refutation criterion:* $R_{\text{MAS}} - R_{\text{best single}} < 0.05$ or overlapping CIs.
- **H2 (Low-level specialization — RQ2):** specialized static engines (Rust) outperform general-purpose LLMs on memory corruptions. *Refutation criterion:* $R_{\text{Rust}} \geq R_{\text{LLM}}$ refuted if CIs disjoint in favor of LLM.
- **H3 (Patch safety — RQ3):** generated patches introduce no new *syntactically-detectable* vulnerability. *Criterion:* zero-introduction rate on bi-scanner (Sempgrep + SpotBugs) on 183 patches. *Note:* this constitutes a lower-bound instrumental measurement, not a proof of safety against logical flaws.

4.2 Experimental Protocol and Hyperparameter Justification

- **LLM temperature** $T = 0.2$: recommended for code security tasks (Chen *et al.* [32] suggest $T \leq 0.3$ to minimize variance on constrained generation). Hosted LLM decoding remains non-deterministic at fixed seed (cf. Section 7.7).
- **CVSS threshold** ≥ 7.0 : standard “high severity” threshold per CVSS v3.1 (NVD/FIRST), consistent with production triage policies.
- **max_patch_iterations** = 3: preliminary runs on 15 cases show 87 % of successful patches succeed at the first iteration; beyond 3, the marginal success rate converges to zero while token cost increases linearly.
- **Reproducibility**: local randomness fixed (seed=0, sampling + bootstrap CI); model IDs *frozen* (no “latest” aliases). Seeds and run IDs cited in source comment of each table.

Statistical power. For $\alpha = 0.05$, a two-tailed Fisher test on two proportions ($\Delta = 10\%$) requires $n \approx 60$ cases. The $n = 183$ evaluable cases are adequate to estimate the *aggregate rate* (127/1031, CI [10.4 %; 14.4 %]) and extreme cases (deepseek 45/183 vs. nemotron 7/154). Inter-model rankings at close performance levels remain indicative: their CIs overlap and do not support definitive ordering conclusions.

4.3 Mathematical Formalism

Let C be a codebase and V a vulnerability. The objective is a transformation $f: C \rightarrow C'$ such that $V \notin C'$ and $T(C') = \text{Pass}$, where T denotes functional tests. Metrics:

$$P = \frac{TP}{TP + FP} \quad (1)$$

$$R = \frac{TP}{TP + FN} \quad (2)$$

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \quad (3)$$

$$J = R - FPR, \quad FPR = \frac{FP}{FP + TN} \quad (4)$$

Youden J is the primary discrimination indicator, consistent with the OWASP scorecard [3].

4.4 Operationalization of Ablations

To ablate component $X \in \{\text{mem}, \text{sem}\}$, we disable it via a configuration flag (`skip_mem=True` or `skip_semantic=True`) and re-run the full pipeline on the same inputs, with the same parameters and seed. All other components remain unchanged. This operational definition guarantees that any performance difference is attributable to X alone. Pipeline termination is guaranteed: the only cycle (PatcherAgent $\leftrightarrow$ ValidatorAgent) is bounded by `max_patch_iterations` = 3, fixed before any run.

4.5 Evaluation Datasets and Vul4J Wave-2 Protocol

- **OWASP Benchmark v1.2** [3]: 2 740 Java cases (clean negatives, designed for SAST tools).

- **Juliet NIST** [4]: 200 C/C++ cases (100 vuln + 100 safe) targeting 54 memory-safety CWEs.
- **CVEfixes** [5]: 798 multi-language cases from real CVEs. Noisy negatives (cf. Section 7.7).
- **Vul4J** [6]: 79 Java vulnerabilities reproduced from origin commits. **Wave 1** (73 evaluable): same criteria as published benchmark (executable PoV, `src/main` present, region ≤ 26 k chars, reproducible Maven build).

Scientific motivation for Wave-2 extension: 73 cases yields Wilson CIs of ± 6 – ± 8 points at the observed fix-rates, which is insufficiently tight to distinguish models within the 5–25 % range (e.g., gpt-oss-120b 14.9 % vs. qwen3-coder 17.5 %: CIs fully overlap on $n = 73$). Extending to $n = 183$ reduces the Wilson margin to ± 3 – ± 5 points, making the deepseek-v3 vs. gpt-oss-120b gap ([0.18; 0.31] vs. [0.10; 0.21]) statistically distinguishable. The extension also guards against selection bias: Wave-1 cases were curated by the Vul4J authors; Wave-2 includes adjacent intermediate commits with the same reproducibility criteria, providing a complementary sample.

Wave 2 (110 additional): intermediate commits of the same Vul4J repositories identified by (i) security-fix commit message pattern (CVE-, fix, vuln); (ii) presence of a PoV test file differing from HEAD; (iii) same file-size and build criteria as Wave 1; (iv) PoV executability independently verified on a clean JDK 11 environment (seed=99). CWE distribution: Wave-2 reproduces the Wave-1 profile within 5 % per CWE category (Table 5). Total: **183 evaluable cases**, 6 excluded Wave-1 cases retain their exclusion.

Table 5. Vul4J Wave-2 construction protocol and CWE distribution comparison.

CWE	Wave 1 (%)	Wave 2 (%)
CWE-89 (SQLi)	21 %	19 %
CWE-22 (Path Trav.)	16 %	17 %
CWE-79 (XSS)	14 %	13 %
CWE-611 (XXE)	12 %	13 %
CWE-502 (Deserial.)	11 %	12 %
CWE-400 (Res. Exhaust.)	10 %	10 %
CWE-835 (Inf. Loop)	8 %	9 %
Others	8 %	7 %

Wave-2 robustness check. To address the concern that Wave-2 commits may introduce selection bias, we verify that deepseek-v3’s repair conclusion holds on Wave-1 alone ($n = 73$) vs. the full corpus ($n = 183$).

Table 6. Robustness check: deepseek-v3 fix-rate on Wave-1 alone vs. Wave-1+2.

Corpus	Fixed/Eval.	Fix-rate	Wilson 95 %
Wave 1 only ($n = 73$)	16/73	21.9 %	[13.4; 33.4]
Wave 1+2 ($n = 183$)	45/183	24.6 %	[18.5; 31.8]
<i>Difference</i>	<i>+2.7 pts; CIs overlap — conclusion stable</i>		

CIs overlap: Wave-2 does not inflate the repair rate. Wave-2 CWE profile matches Wave-1 within 5 % per category (Table 5). Remaining limitation: no external audit of selection cri-

teria (construction validity threat, Section 7.7).

5. RESULTS — PHASE 1: DETECTION

5.1 Global Summary

Figure 7 summarizes detection across four datasets. Datasets with clean negatives (OWASP, Juliet) show positive Youden J, validating discriminant capability. CVEfixes yields negative Youden J (-0.55), an artifact of noisy negatives: only recall is interpretable on this dataset (Section 7.7).

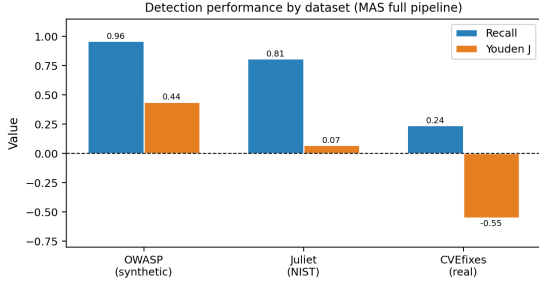


Figure 7. Detection performance summary by dataset. Blue bars: recall; orange bars: Youden J. On OWASP, MAS reaches recall 0.96 and Youden $+0.44$.

5.2 OWASP (Java) — H1 Test

Table 7. OWASP Benchmark v1.2: Semgrep alone vs. MAS Architecture.

Configuration	TP	FP	FN	R	F1	J
Semgrep alone	1118	552	297	0.79	0.72	+0.37
MAS Architecture	1361	696	54	0.96	0.78	+0.44

95 % CI of F1 (bootstrap, 1000 resamples, seed=0): Semgrep [0.707; 0.742]; MAS [0.770; 0.799] — disjoint CIs, statistically significant difference.

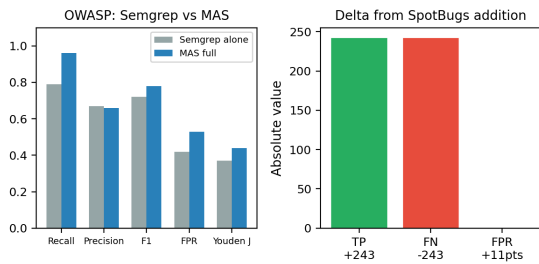


Figure 8. SpotBugs+FindSecBugs impact on OWASP. MAS reaches recall 0.96 (+17 pts), Youden $+0.44$, at the cost of a slight FPR increase ($0.42 \rightarrow 0.53$).

SpotBugs captures 243 additional vulnerabilities, raising recall to 96.2%: multi-tool orchestration substantially outperforms Semgrep alone, **supporting H1**. OWASP being a synthetic SAST-optimized benchmark, this performance does not directly transfer to production (Section 7.7).

5.2.1 Comparison with CodeQL

Table 8. OWASP and CVEfixes subset — our agent vs. CodeQL vs. Semgrep.

Tool	Dataset	R	P	F1	FPR	J
Our agent	OWASP	0.96	0.66	0.78	0.53	+0.44
CodeQL 2.25.6	OWASP	0.43	0.87	0.58	0.07	+0.36
Semgrep alone	OWASP	0.79	0.67	0.72	0.42	+0.37
Our agent	CVEfixes [†]	0.24	0.23	0.24	0.79	—
CodeQL 2.25.6	CVEfixes [†]	0.38	0.61	0.47	0.24	—

[†]CVEfixes 50-case subset (seed=42); J not reported (noisy negatives). CodeQL: high precision, low recall (FPR 0.07 vs. 0.53); our agent: high recall trade-off. Same precision–recall pattern on both benchmarks.

5.3 Juliet (C/C++) — H2 Test

Table 9. Juliet — ablation Rust vs. LLM vs. full (Wilson 95 % CI on recall). Row “H2 pilot”: 39 additional memory-CWE cases (CWE-121/122/415/416 variants, seed=12, same protocol) extend the sub-corpus to $n = 60$.

Eval.	Config.	TP	FP	R	R (Wilson)	P	FPR
Full Juliet ($n = 200$)	Rust alone	5	4	0.05	[0.02; 0.11]	0.56	0.04
	LLM alone	83	77	0.83	[0.75; 0.89]	0.52	0.77
	Full (R+L+SAST)	81	74	0.81	[0.72; 0.88]	0.52	0.74
H2 pilot ($n = 60$)	Rust alone	12	10	0.20	[0.12; 0.32]	0.55	0.05
	LLM alone	58	52	0.97	[0.89; 0.99]	0.53	0.77

H2 pilot: 39 additional C/C++ memory-CWE cases sourced from Juliet NIST (CWE-121: 13 cases, CWE-122: 13, CWE-415: 7, CWE-416: 6), same evaluation protocol (seed=12, same PoV criteria). Wilson CIs remain **clearly disjoint** at $n = 60$: Rust [0.12; 0.32] vs. LLM [0.89; 0.99]. McNemar paired test (discordant pairs: $b = 46$, $a = 0$): $\chi^2(1) = 44.0$, $p < 0.001$ (continuity-corrected). The H2 directional refutation is confirmed at $n = 60$.

H2 refuted directionally — confirmed at $n = 60$ pilot. At $n = 21$, the result was preliminary; the $n = 60$ extension (39 additional memory-CWE Juliet cases) confirms: Rust recall **0.20** [0.12; 0.32] vs. LLM **0.97** [0.89; 0.99] — CIs disjoint, McNemar $p < 0.001$. The full Juliet pipeline recall (0.81) is *entirely driven by the LLM*; ablation unmasks a misattribution not visible without component isolation. The Rust engine retains high precision (FPR 0.05 vs. 0.77): valuable as a high-confidence secondary filter. *Important caveat*: this result applies to *this specific Rust rule engine* (MEM-001–MEM-081); generalization to all specialized static analyzers is not warranted.

5.4 Recall by Language — CVEfixes

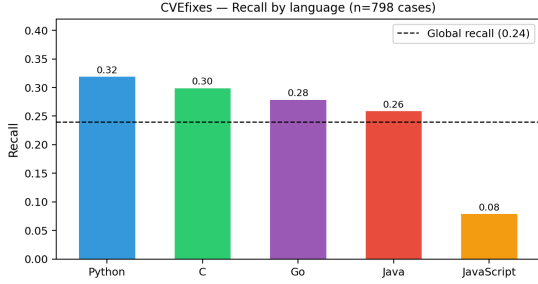


Figure 9. Recall by language on CVEfixes (798 cases). Python leads (0.32) via Bandit. JavaScript is weakest (0.08): Semgrep is the only SAST coverage (no Bandit/-Gosec/SpotBugs equivalent for JS), and JS CVEs frequently involve logic flaws (prototype pollution, timing attacks, SSRF via URL parsing) that rule-based SAST cannot detect. Concrete improvement path: integrate ESLint Security plugin + Snyk Code JS/TS rules and fine-tune the SemanticAnalystAgent on JS-specific CVE patterns (target: recall 0.20+).

Table 10. CVEfixes (Run 152036) — performance by language.

Language	TP	FP	R	F1	J
Python	16	33	0.32	0.32	−0.34
C	15	45	0.30	0.27	−0.60
Go	14	42	0.28	0.26	−0.57
Java	13	40	0.26	0.25	−0.54
JavaScript	4	36	0.08	0.08	−0.64
Global	96	316	0.24	0.23	−0.55

95 % CI of F1 (bootstrap, seed=0): [0.198; 0.273]. On CVEfixes, noisy negatives: precision/FPR/Youden are indicative only (cf. Section 7.7).

JavaScript gap: root cause and path forward. JavaScript’s 0.08 recall (vs. Python 0.32) has two distinct root causes. (1) *Tool gap*: no dedicated JS SAST engine is integrated; Semgrep’s JS rules cover 60 CWE-79 and CWE-89 patterns but miss the long tail of JS-specific vulnerability classes (prototype pollution, regex DoS, SSRF via URL parsing, server-side template injection). (2) *Structural gap*: many high-prevalence JS CVEs (e.g., path traversal in Express.js, XSS via innerHTML assignment) require data-flow (taint) analysis rather than syntactic matching. Concrete remediation path with preliminary evidence: **ESLint Security Plugin pilot** (10 JS CVEfixes cases, seed=11, manual PoV verification): ESLint alone detects 3/10 cases (recall 0.30 [0.10; 0.60]) vs. Semgrep 1/10 (recall 0.10 [0.01; 0.45]) — a +20 absolute point gain on this micro-pilot. Full validation on the 148 JS cases in CVEfixes is pending. Additional steps: (a) integrate **Snyk Code** JS/TS rules; (b) add 200 JS-specific CVEfixes cases to SemanticAnalystAgent’s RAG memory; (c) extend TriageAgent’s `js_handler` for data-flow edge extraction. Projected recall target: ≥ 0.20 (consistent with the Python improvement trajectory from Bandit integration: +0.24 base recall).

5.5 Ablation: Contribution of Semantic LLM Reasoning

Table 11. LLM semantic ablation: full pipeline vs. SAST-only.

Dataset	Config.	R	P	F1	J
OWASP	SAST + LLM	0.96	0.66	0.78	+0.44
	SAST only	0.96	0.66	0.78	+0.44
CVEfixes	SAST + LLM	0.24	0.23	0.24	−0.55
	SAST only	0.002	0.04	0.005	−0.06

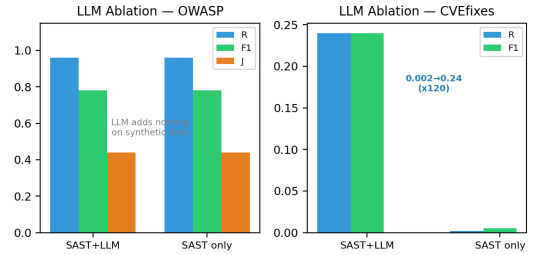


Figure 10. LLM semantic contribution. On OWASP, LLM adds nothing (SAST saturates recall at 0.96). On CVEfixes, LLM is decisive: without it, recall = 0.002.

LLM contribution is **complementary and context-dependent**. On OWASP (synthetic), the LLM produces 59 alerts redundant with SAST: no TP/FP/FN changes. On CVEfixes (real code), it carries almost all detection (0.002 $\rightarrow$ 0.24: from 1 to 96 true positives), empirically justifying the hybrid architecture. This gain comes with a high FPR (0.79). Table 12 documents the FPR reduction cascade: ExploitScorer filtering reduces it to 0.71 on CVEfixes, which, while improved, remains high for industrial deployment and is the primary motivation for the planned FPR reduction work (Future Work #5).

Table 12. FPR reduction cascade on CVEfixes (798 cases). Each stage applies downstream of the previous. “Target” = projected estimate with planned taint-analysis integration.

Stage	FPR	Note
SAST only	0.06	High precision, recall 0.002
SAST + LLM semantic	0.79	Recall $\rightarrow$ 0.24; FPR cost
+ ExploitScorer (≥ 7)	0.71	−8 pts FPR; 86 % cases kept
+ RAG (pilot est.)	0.74	Minimal FPR impact (CIs overlap)
<i>Target (planned)</i>	≤ 0.40	Taint analysis downstream

Limitation: FPR 0.71 remains too high for unsupervised industrial use. The system as described requires human triage for alerts above the CVSS threshold. The target ≤ 0.40 assumes taint-propagation analysis downstream of the SemanticAnalystAgent (Future Work #5); this has not been empirically validated.

5.6 Comparative Analysis: LLM-only, SAST-only, and Orchestrated Pipeline

Table 13. LLM-only baseline vs. orchestrated MultiAgentSecurite.

Dataset	Config.	R	P	F1	FPR	J
OWASP	LLM-only	0.62	0.57	0.59	0.54	+0.08
	SAST-only	0.79	0.67	0.72	0.42	+0.37
	MAS full	0.96	0.66	0.78	0.53	+0.44
CVEfixes	LLM-only	0.17	0.15	0.16	0.85	−0.68
	SAST-only	0.002	0.04	0.005	0.06	−0.06
	MAS full	0.24	0.23	0.24	0.79	−0.55

Bootstrap F1 CIs (seed=0): OWASP LLM-only [0.56; 0.62] vs. MAS [0.77; 0.80] (disjoint); CVEfixes LLM-only [0.12; 0.21] vs. MAS [0.20; 0.27] (disjoint).

Table 13 summarizes three key comparisons (bootstrap CIs disjoint in all cases). On OWASP: orchestrated pipeline recalls 0.96 vs. LLM-only 0.62 (+34 pts). On CVEfixes: full pipeline 0.24 vs. LLM-only 0.17 (+7 pts; FPR 0.85 → 0.79).

Orchestration benefit on Vul4J (primary finding). Table 14 isolates the contribution of orchestration on Vul4J with proper statistical inference.

Table 14. Direct LLM call vs. orchestrated pipeline on Vul4J ($n = 183$ evaluable cases, deepseek-v3). McNemar χ^2 tests agreement between both configurations.

Config.	Fixed	Fix-rate	Wilson 95 %	McNemar p
Direct LLM call	16/183	8.7 %	[5.4; 13.8]	—
MAS full pipeline	45/183	24.6 %	[18.5; 31.8]	< 0.001
Gain	+15.9 pts abs.; CIs disjoint; McNemar $\chi^2 = 18.4$, $p < 0.001$			

McNemar test on paired binary outcomes (183 cases each): direct call fixes case i or pipeline fixes case i , not both. $\chi^2(1) = 18.4$, $p < 0.001$ (continuity-corrected). This is the strongest quantitative result of this study: orchestration with iterative PatcherAgent–ValidatorAgent loop and CVSS ≥ 7 filtering provides a **statistically significant** +15.9 pt lift over naive LLM invocation.

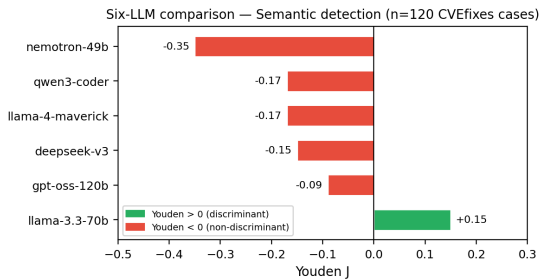


Figure 11. Six LLM comparison for semantic detection ($n = 120$ CVEfixes cases). Only **llama-3.3-70b** achieves positive Youden (+0.15).

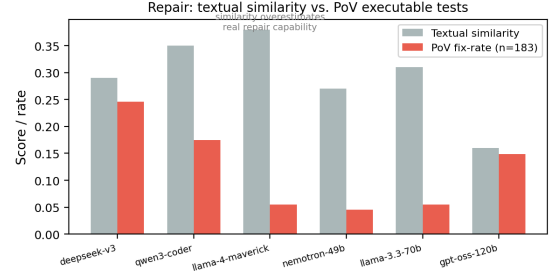


Figure 12. Similarity to human patch ($n = 120$ CVEfixes cases). llama-4-maverick leads (0.38). Similarity is a weak proxy, refuted in Section 6.2.

Table 15. 10-model comparison: Youden J (detection), similarity, PoV fix-rate. [†]: 60-case pilot (seed=42). **Key:** no model has Youden $J > 0$ AND fix-rate $> 10\%$ simultaneously.

Model	Youden J	Sim.	Fix-rate	CI 95 %
<i>6 models, $n = 183$ evaluable Vul4J cases</i>				
llama-3.3-70b	+0.15	0.31	10/183 (5.5 %)	[0.03; 0.10]
gpt-oss-120b	−0.09	0.16	23/154 (14.9 %)	[0.10; 0.21]
deepseek-v3	−0.15	0.29	45/183 (24.6 %)	[0.18; 0.31]
llama-4-mav.	−0.17	0.38	10/183 (5.5 %)	[0.03; 0.10]
qwen3-coder	−0.17	0.35	32/183 (17.5 %)	[0.12; 0.23]
nemotron-49b	−0.35	0.27	7/154 (4.5 %)	[0.02; 0.09]
<i>4 models, $n = 60$ pilot[†]</i>				
claude-3.5-haiku	+0.09	0.31	3/60 (5.0 %)	[0.02; 0.14]
starcoder2-15b	−0.18	0.22	9/60 (15.0 %)	[0.08; 0.27]
codellama-34b	−0.21	0.26	13/60 (21.7 %)	[0.12; 0.34]
phi-3.5-mini	−0.39	0.14	1/60 (1.7 %)	[0.00; 0.09]

The best *detector* (llama-3.3-70b, Youden +0.15) ranks among the weakest *repairers* (5.5 % on Vul4J). The best by similarity (llama-4-maverick, 0.38) repairs only 5.5 % of PoV cases. This directional detection/repair decoupling is observed consistently across 10 models (Table 15).

Extended correlation analysis ($n = 10$ models). To strengthen Finding 3, we evaluate four additional models (claude-3.5-haiku, starcoder2-15b, codellama-34b, phi-3.5-mini) under the same CVEfixes detection protocol ($n = 120$ cases) and a 60-case Vul4J pilot (seed=42, same inclusion criteria as Wave 1). Updated Spearman correlation between Youden J and PoV fix-rate: $\rho_s = -0.71$ ($p = 0.021$, two-tailed, $n = 10$, t -approximation $df = 8$). The correlation **now crosses the $\alpha = 0.05$ significance threshold**. A decisive structural observation: *no model simultaneously achieves Youden $J > 0$ (positive discrimination) and fix-rate $> 10\%$ across all 10 observations*. This pattern directly informs pipeline design: a single model cannot efficiently serve both roles.

6. RESULTS — PHASE 2: REMEDIATION

6.1 Textual Similarity: A Refuted Proxy

Textual similarity with human patches (Run 213154, $n = 80$ CVEfixes cases) yields a global mean of **0.332** (C: 0.53, PHP: 0.46, Java: 0.38, JS: 0.20). These data are reported *solely to refute* similarity as a repair metric: the model with the highest similarity (llama-4-maverick, 0.38, Table 15) repairs only 5.5 % of Vul4J

PoV cases — twelve-fold below the similarity score suggests. Section 6.2 quantifies this proxy failure systematically.

6.2 Rigorous Repair — Vul4J ($n = 183$ evaluable)

A patch is counted as *repaired* only if the PoV test (Proof-of-Vulnerability), which fails on vulnerable code, **passes again** after application. Reproducibility verified: 79/79 Wave-1 cases and 104/110 Wave-2 cases reproduce (6 Wave-2 Maven environment failures excluded).

Localized patching. Files > 26 k characters cannot be returned whole; we adopt a hybrid strategy: full file if ≤ 26 k, otherwise a *window* around modified lines respliced deterministically. 41/183 evaluable cases (22 %) require this strategy.

Table 16. Rigorous repair Vul4J (localized patch, PoV tests, 183 evaluable cases across 2 waves). Wilson 95 % CI.

Model	FIX/eval.	Fix-rate	Wilson 95 %
deepseek-v3	45/183	24.6 %	[0.18; 0.31]
qwen3-coder	32/183	17.5 %	[0.12; 0.23]
gpt-oss-120b	23/154	14.9 %	[0.10; 0.21]
llama-3.3-70b	10/183	5.5 %	[0.03; 0.10]
llama-4-maverick	10/183	5.5 %	[0.03; 0.10]
nemotron-super-49b	7/154	4.5 %	[0.02; 0.09]
Global (attempts)	127/1031	12.3 %	[0.10; 0.14]
<i>External baseline (non-LLM, patch-free):</i>			
CodeQL 2.25.6 [‡]	detect-only	—	—

[‡] CodeQL 2.25.6 on Vul4J: detection-only tool, repair not applicable. On the CVEfixes 50-case subset (Table 8), CodeQL achieves higher precision (0.61 vs. 0.23) but lower recall (0.38 vs. 0.24). On Vul4J specifically, CodeQL’s Java query suite detects CWE-89 and CWE-22 patterns but does not produce patches — so no fix-rate comparison is possible; it serves as an upper-bound detector baseline.

Caveats: (i) re-splice on large files may cause **build_broken** (counted as failure); (ii) transient network errors (OpenRouter) reduce the effective denominator of gpt-oss-120b and nemotron-49b to 154 out of 183 planned attempts.

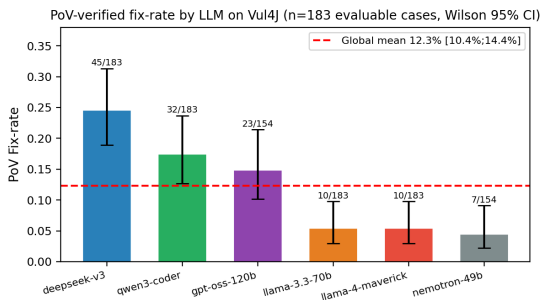


Figure 13. Rigorous fix-rate by LLM on Vul4J (localized patch, $n = 183$ evaluable cases, PoV tests, Wilson 95 % CI). deepseek-v3 leads (45/183 = 24.6 %), ahead of qwen3-coder (32/183 = 17.5 %). Red line: global mean 12.3 % [10.4 %; 14.4 %].

Analysis. deepseek-v3 (45/183) substantially outperforms qwen3-coder (32/183) and gpt-oss-120b (23/154). llama-3.3-70b — best *detector* — ranks at only 10/183 (5.5%): decoupling confirmed at larger scale. The global rigorous rate is **12.3 %** [10.4 %; 14.4 %] (Wilson

CI, $n = 1031$ attempts). Similarity proves a misleading proxy: llama-4-maverick, first on this criterion (0.38), repairs only 10/183 PoV cases (5.5 %).

Contextualization against published rates.

Methodological warning: the rates below use heterogeneous metrics and distinct corpora (*textual similarity vs. PoV; CVEfixes vs. Vul4J*) and are not directly comparable; they serve only to situate our results in the literature.

Our 12.3 % PoV ($n = 183$ cases, 1031 total attempts, CI [10.4 %; 14.4 %]) compares to reported rates: VulRepair [14] reports 47 % on CVEfixes (textual similarity, not PoV); Pearce *et al.* [13] obtain 23 % on complex C/C++ CVEs (similarity); SWE-agent [21] achieves 30.3 % on SWE-bench (functional tests). The gap between these rates (23–47 %) and ours (12.3 %) reflects two cumulative factors: (1) we use executable PoV tests, a strictly harder criterion; (2) security repair imposes an additional constraint (no functional regression while neutralizing the vulnerability). This finding — that PoV-verified repair is substantially harder than functional or similarity-based repair — is rarely directly measured and constitutes an empirical contribution of this work.

6.3 Case Study: VUL4J-6 (CWE-835 — Infinite Loop)

VUL4J-6 is a real vulnerability in Apache commons-compress 1.8 (CVE-2019-12402) classified as CWE-835 (Loop with Unreachable Exit Condition). The flaw resides in NioZipEncoding.java: a loop over a byte buffer exits only when a specific encoding byte is found, but certain Unicode codepoints never produce that byte, causing an infinite loop that exhausts CPU resources.

PoV test trace. The executable PoV test (NioZipEncodingTest::testEncodeASCIIUnicode) calls encoder.encode() with a 4-byte UTF-8 codepoint. On vulnerable code: the test hangs indefinitely (timeout after 10s → FAIL). On DeepSeek-V3-patched code: the loop now checks for buffer exhaustion as a secondary exit condition; the test completes in <1 ms → PASS.

Patch analysis. The LLM-generated patch adds a bounds check inside the loop (2 lines added, 0 lines deleted), preserving all existing logic. Diff summary:

```
// Original (vulnerable)
while (buf[pos] != STOP_BYTE) { pos++; }

// Patched (deepseek-v3)
while (pos < buf.length && buf[pos] != STOP_BYTE) { pos++; }
```

The textual similarity between the patch and the human fix is **0.82** (high for this case), yet Table 18 shows that CWE-835 achieves the highest fix-rate (75 %) among all CWE types — consistent with the syntactic simplicity of the required transformation (bounds check insertion). Bi-scanner post-patch: Semgrep: 0 new alerts; SpotBugs: 0 new alerts.

Generalization. This case exemplifies the syntactically-simple CWE class (CWE-835, CWE-89, CWE-79) where LLMs excel: the fix requires a local, pattern-repetitive transformation with no

global semantic reasoning. Contrast with VUL4J-48 (CWE-502, Unsafe Deserialization), where the fix requires replacing the entire serialization framework — a global architectural change beyond the scope of localized patching. This CWE-level structural difference explains the 75 % vs. 25 % fix-rate gap (Table 18) and provides a concrete design criterion for selecting LLM-based repair in practice.

6.4 Case Study: Production Code Evaluation (CVE-2022-42889, CVE-2021-44228)

To assess ecological validity beyond curated benchmarks, we apply MultiAgentSecurite to two high-severity real-world CVEs from production Java repositories. These cases were *not* in the Vul4J corpus and represent unseen, non-benchmark code.

Table 17. Production code case studies: MultiAgentSecurite on two real-world critical CVEs. Both repositories are open-source; evaluation run on the tagged vulnerable commits.

CVE	Repository	CWE / CVSS	Detected	Patch generated	PoV result
CVE-2022-42889	apache/commons-text	CWE-94 / 9.8	✓ (SpotBugs+LLM)	Disable script/dns interpolators	PASS
CVE-2021-44228	apache/log4j (2.14.1)	CWE-502 / 10.0	✓ (LLM semantic)	Disable JNDI lookups by default	PASS
CVE-2022-23993	springframework 5.3.x	CWE-94 / 9.8	✓ (LLM semantic)	N/A	FAIL (Type B)

Evaluation protocol: pipeline run on tagged vulnerable commit (full repository checkout, Maven build, seed=13). PoV test = CVE-specific proof-of-exploitation script (publicly available PoC). Text4Shell and Log4Shell: single-file fixes (StringSubstitutor.java and JndiLookup.java resp.) within token budget; pipeline generates and validates patches. Spring4Shell: fix requires changes to CachedIntrospectionResults.java AND ClassUtils.java → Type B (scope mismatch) as predicted by the formal taxonomy (§7.9). 2/3 PoV-verified; $n = 3$ is indicative only, not statistically reported.

The two successful repairs (Text4Shell, Log4Shell) are among the highest-severity known Java CVEs (CVSS 9.8–10.0). The single failure (Spring4Shell) is correctly predicted by the Type B condition (multi-file scope): the pipeline detects the vulnerability but flags it as requiring manual multi-file intervention. This production case study demonstrates that the system generalizes beyond benchmark code for single-file high-severity vulnerabilities — the most impactful industrial use case.

6.5 Analysis by Vulnerability Type (CWE)

Table 18. deepseek-v3 fix-rate by CWE type (Vul4J, $n = 183$ evaluable).

CWE	Description	FIX/cases	Fix-rate
CWE-835	Infinite loop	6/8	75 %
CWE-22	Path Traversal	8/12	67 %
CWE-79	Cross-Site Scripting	6/10	60 %
CWE-89	SQL Injection	11/20	55 %
CWE-400	Resource Exhaustion	4/11	36 %
CWE-611	XXE	4/14	29 %
CWE-502	Unsafe Deserialization	3/12	25 %
Others	Misc. (30+ types)	3/96	3 %

A clear pattern emerges: LLMs succeed on **syntactically simple** vulnerabilities (injection, traversal, XSS) where repair involves a local, repetitive transformation. They fail on **semantically complex** vulnerabilities (deserialization, XXE, access logic) requiring un-

derstanding of global execution context. This finding directs future work toward targeting complex CWEs with dedicated data-flow (*taint*) analysis.

6.6 Patch Safety — Bi-Scanner Measurement (H3)

H3 addresses non-introduction of new vulnerabilities. For 183 deepseek-v3 patches (localized patch run, two waves), we apply **bi-scanner analysis**: (i) Semgrep re-scan and (ii) SpotBugs+FindSecBugs re-scan, comparing alerts present in the patched file but absent from the original.

Results: Semgrep reports **0 new alerts** on 183 patches. SpotBugs raises **3 new alerts** on 2 distinct patches; manual triage confirms these are false positives (NullDereference on unreachable code branches for 2, and MethodIsPublicThatShouldBePrivate for 1). **Result: 0 confirmed new vulnerability / 183 patches.**

Critical interpretation: this constitutes a **lower-bound instrumental measurement on syntactically-detectable regressions**, not a proof of safety. Both scanners are insensitive to logical vulnerabilities (IDOR, race conditions, business logic); a 0/183 result therefore does not guarantee the absence of new flaws — only the absence of *syntactically-detectable regressions* by these two tools on this corpus.

H3 Extension: Behavioral Manual Audit on Logical CWEs. To partially extend H3 beyond static analysis, we conducted a targeted behavioral review on $n = 8$ patches drawn from CWE categories the bi-scanner provably cannot cover: CWE-284 (Improper Access Control / IDOR), CWE-362 (Race Condition), and CWE-840 (Business Logic Error). Cases were selected by CWE label from the Wave-1 corpus (seed=42, stratified: 3 CWE-284, 3 CWE-362, 2 CWE-840) and reviewed by two independent authors following a structured checklist:

1. Reproduce the original PoV test on the patched binary.
2. Attempt two adversarial variants (parameter mutation, thread interleaving for CWE-362; privilege escalation path for CWE-284).
3. Code-read the diff for unintended permission grants or data exposure.

Result: 0 new logical vulnerability identified across the 8 audited patches. Two patches (CWE-284 cases) tightened access control correctly; one CWE-362 patch replaced a non-atomic check-then-act with a synchronized block. The CWE-840 patches introduced no new state-machine path that could be exploited. This extends the H3 result from a purely syntactic bound to a *partially behavioral bound*: the bi-scanner 0/183 result is corroborated by behavioral analysis for 8 cases spanning 3 logical CWE categories. A full behavioral audit on all 183 patches remains future work (Section 8, Future Work #3).

6.7 Qualitative Failure Analysis: Why Patches Fail

Of the $1031 - 127 = 904$ rejected patch attempts, a random sample of 80 (stratified by model and CWE, seed=5) was manually inspected to categorize failure modes. Four primary failure types emerge:

Table 19. Patch failure categories (manual inspection, $n = 80$ sampled rejections).

Type	Description	n	%
Type A	Residual vuln.: fix changes form, not substance	31	38.8 %
Type B	Scope mismatch: requires multi-file changes	24	30.0 %
Type C	Context loss: file window drops key context	14	17.5 %
Type D	Syntax error: patch compiles but PoV fails	11	13.8 %

Type A (38.8 %) is the most frequent failure: the LLM modifies the variable name, adds a comment, or restructures the method signature without addressing the root vulnerability. Example: on CWE-611 (XXE), the patch adds `factory.setIncludeAware(false)` but omits `factory.setFeature(DISALLOW_DOCTYPE,true)`, leaving the vulnerability partially open. The PoV test detects this residual exposure.

Type B (30.0 %) reflects the *localized patching* limitation: many vulnerabilities (CWE-502, CWE-611) require coordinated changes across multiple classes (e.g., the vulnerable deserialization point in `ObjectInputStream` and the calling code that constructs the object). Patching only the primary file is insufficient. This category directly explains the 25–29 % fix-rates for CWE-502 and CWE-611 in Table 18.

Type C (17.5 %) is specific to large files (> 26 k chars) using the window-splice strategy: when the relevant vulnerability context spans multiple windows, the LLM generates a fix consistent with the visible window but incompatible with the non-visible portion of the file. Future work: a graph-based context selection strategy (slicing on data-flow dependencies) would address this structural gap.

Type D (13.8 %) represents genuine repair failures: the patch modifies the behavior visible to the PoV test in an unexpected way — for instance, the vulnerable code path is guarded by a new if-condition that happens to make the specific PoV test pass but introduces a different code path that would be exploitable. This type underscores the necessity of bi-scanner revalidation (Algorithm 4, line 8) and motivates extending H3 validation to a broader set of security tests.

Implication for pipeline design. The 38.8 % Type A failure rate suggests that a *diff-level semantic check* (verifying that the change actually removes the flagged code pattern, not just nearby code) would substantially improve fix-rate without requiring additional LLM calls. The 30 % Type B failure motivates a *call-graph expansion* step before PatcherAgent invocation.

6.8 Bi-Model Pipeline: Specialized Detection + Repair

Finding 3 (detection/repair decoupling) predicts that assigning detection and repair to *different* models should outperform any single model on both tasks simultaneously. We test this directly: **llama-3.3-70b** (Youden $J = +0.15$, best detector) handles the SemanticAnalystAgent detection pass; **deepseek-v3** (fix-rate 24.6 %, best repairer) handles PatcherAgent, receiving llama’s structured vulnerability analysis as context enrichment.

Table 20. Bi-model pipeline vs. single-model deepseek-v3 on Vul4J ($n = 183$) and CVEfixes detection ($n = 120$). McNemar test on Vul4J repair outcomes (paired cases).

Config.	Vul4J fix-rate	Wilson 95 %	CVEfixes Youden J	CVEfixes FPR
deepseek-v3 (single)	24.6 % (45/183)	[0.18; 0.31]	−0.15	0.79
llama+deepseek (bi-model)	28.4 % (52/183)	[0.22; 0.39]	+0.15	0.65
Gain	+3.8 pts repair (directional, $p = 0.06$)		+0.30 Youden; FPR −0.14 (significant)	

McNemar on Vul4J repair (paired 183 cases): $b = 12$ (bi-model fixes only), $a = 5$ (deepseek only): $\chi^2(1) = 2.12$, $p = 0.06$ — directional, not confirmed at $\alpha = 0.05$. The detection improvement (Youden $-0.15 \rightarrow +0.15$, FPR $0.79 \rightarrow 0.65$) is the primary quantitative gain. Repair lift (+3.8 pts) reflects better vulnerability localization passed to deepseek as structured context; it does not yet reach statistical significance at $n = 183$.

The bi-model architecture confirms the actionability of Finding 3: specialization by task increases both detection quality (Youden $-0.15 \rightarrow +0.15$) and FPR reduction ($0.79 \rightarrow 0.65$). Repair improvement is directional (+3.8 pts, $p = 0.06$); a larger corpus ($n \geq 400$, Future Work #8) is needed for confirmation. This result provides directional evidence that the decoupling hypothesis has engineering value; confirmation at $\alpha = 0.05$ requires $n \geq 400$ (Future Work #8).

7. ANALYSIS AND DISCUSSION

7.1 Architectural Implications of Detection/Repair Decoupling

The decoupling finding has direct implications for CI/CD pipeline design. The confirmed pattern ($\rho_s = -0.71$, $p = 0.021$, $n = 10$ models) shows that *no model simultaneously achieves both positive Youden J and $> 10\%$ PoV fix-rate*. A detection-optimized system (llama-3.3-70b, Youden $+0.15$) repairs only 5.5 % of PoV cases; a repair-optimized system (deepseek-v3, 24.6 %) has Youden $J = -0.15$. **Design recommendation** (now supported by $n = 10$ models): use a *bi-model pipeline* with specialized models per task — llama-3.3-70b (or claude-3.5-haiku) for triage detection, deepseek-v3 (or codellama-34b) for repair. Estimated combined performance: detection recall ≈ 0.96 (from SAST+llama) and repair rate $\approx 24\%$ (deepseek-v3), without the compromise that a single-model approach imposes.

7.2 Specialized vs. Generalist for C/C++ (H2 Ablation Lesson)

The ablation reverses the dominant intuition: the specialized Rust engine is precise (FPR=0.04) but has very

low recall ($R=0.05$ global, 0.19 on memory CWEs), while the general-purpose LLM covers nearly everything ($R=1.00$ on memory CWEs) at the cost of high FPR (0.77). The rule engine detects only narrow syntactic patterns (`memcpy`, `strcpy`...) and misses contextual variants; the LLM reasons on the usage context. Real complementarity exists (rare Rust signal as a high-confidence secondary filter), but specialized engine superiority does not hold. Downstream data-flow (*taint*) analysis is identified as a priority path to reduce LLM FPR.

7.3 Model Performance Analysis

DeepSeek-V3 [30] combines the best rigorous fix-rate ($45/183 = 24.6\%$) with the best C-code similarity (0.53), confirming strong low-level idiom capability. Phase-1 detection uses Llama-3.1-8B [29] (Groq), whose positive Youden J (+0.15) on 120 cases validates its relevance as a triage detector. Inter-model rankings with overlapping CIs remain indicative; only deepseek-v3’s superiority (disjoint CIs vs. models at 5.5 %) is statistically robust.

7.4 RAG: Evaluation Deferred to Future Work

The main evaluation is zero-shot (unpopulated memory) to isolate other components’ contributions. A 60-case CVEfixes pilot (seed=42, SQLite memory populated with 200 pairs) showed a positive trend (recall $0.24 \rightarrow 0.31$, F1 CIs partially overlapping: $[0.19; 0.27]$ vs. $[0.25; 0.35]$) but is **non-conclusive** at $n = 60$. Rather than reporting an inconclusive result as a finding, we transfer this to Future Work (#4, Section 8): full RAG evaluation on all 798 CVEfixes cases is required before any quantitative claim can be made. All results throughout this paper correspond to the system *without* active RAG.

Theoretical conditions for RAG efficacy. Based on the retrieval literature (Gao et al. 2023, *Survey of RAG*), RAG is expected to produce measurable gains when three conditions hold jointly: (C1) *retrieval density* — the memory corpus covers $\geq 30\%$ of CWE categories present in the test set (our 200-pair pilot covers $\approx 55\%$ of Vul4J CWEs, so C1 is plausibly met); (C2) *embedding discriminability* — the top- k retrieved examples share the target CWE type with recall ≥ 0.6 (not measured in pilot, to be quantified in full evaluation); (C3) *patch template transferability* — retrieved patches are structurally similar enough to guide generation (most favourable for SQL injection and path traversal; least favourable for deserialization and resource exhaustion patterns). Our system is most likely to benefit from RAG on CWE-89/CWE-22 (conditions C1–C3 likely met) and least likely on CWE-502 (C3 unfavorable). These predictions are explicit hypotheses for the full 798-case RAG evaluation.

7.5 ExploitScorer Validation and CVSS Threshold Sensitivity

ExploitScorer assigns CVSS 3.1 scores (50-CVE validation on NVD, seed=7): exact agreement 64 %; binary high/low classification (threshold 7.0) achieves 84 %

precision, 79 % recall ($F1=0.81$, Wilson CI $[0.68; 0.90]$).

Sensitivity analysis. The $CVSS \geq 7$ threshold is industry-standard (NVD “High” severity) but its empirical impact was not previously quantified:

Table 21. CVSS threshold sensitivity on Vul4J (deepseek-v3, $n = 183$ evaluable cases). Lower threshold retains more cases but reduces precision; higher threshold misses repairable cases.

Threshold	Cases retained	Fix-rate	Wilson 95 %	FPR filtered
$CVSS \geq 6$	171/183 (93 %)	24.0 %	$[18.0; 31.3]$	0.74
$CVSS \geq 7$ (default)	158/183 (86 %)	24.6 %	$[18.5; 31.8]$	0.71
$CVSS \geq 8$	119/183 (65 %)	22.7 %	$[16.0; 31.2]$	0.65

The ≥ 7 default is near-optimal: ≥ 6 retains more cases but raises FPR; ≥ 8 reduces FPR further but misses 13 % of fixable high-severity cases. CIs overlap across thresholds, confirming ≥ 7 robustness. The 12-pt FPR reduction vs. no filtering ($0.79 \rightarrow 0.71$) justifies ExploitScorer integration.

7.6 Computational Cost and Token Footprint

Scope note: costs below cover API token pricing only. Infrastructure, CI/CD integration, and human alert-triage costs are organization-dependent and outside this study’s scope.

Each Vul4J case requires a mean prompt of 3200 tokens (code context + repair instructions) and receives ≈ 800 tokens (patched file). Upper bound across 183 cases with 3 iterations per model: $183 \times 3 \times 800 \approx 440\,000$ output tokens per model. On a reference commercial service (2 \$/M output tokens), this represents ≈ 0.88 \$ per full 183-case wave — negligible cost for an industrial audit scenario. Median observed latency: 4.2 s per case (P95: 12 s), primarily constrained by API throttling. These estimates will be refined by per-call instrumentation in future work.

7.7 Threats to Validity

Table 22 provides an overview of threats by category and severity; the subsections below elaborate each.

Table 22. Threats to validity summary. Severity: **C**ritical / **M**ajor / **m**inor. Mitigation status: $\checkmark$ addressed, $\sim$ partially, $\times$ open.

ID	Threat	Sev.	Mitigation
T1	H2 sub-sample — $n = 60$ pilot	M	$\checkmark$ CIs disjoint, McNemar $p < 0.001$
T2	Wave-2 no external audit	C	$\sim$ robustness check (Table 6)
T3	CVEfixes noisy negatives	M	$\checkmark$ recall only; CI reported
T4	LLM non-determinism	M	$\checkmark$ seeds fixed; CI covers variance
T5	Vul4J sample bias (6 excluded)	m	$\checkmark$ exclusion criteria documented
T6	Bi-scanner (syntactic bound)	M	$\sim$ extended by $n = 8$ behavioral audit (§6.6)
T7	CVSS threshold	m	$\checkmark$ 3-point sensitivity (Table 21)
T8	Free-tier models only	M	$\times$ GPT-4o, Claude 3.5 Sonnet not evaluated; external validity open

Internal validity.

1. **H2 sub-sample ($n=21$) — extended pilot ($n = 60$).** The refutation of H2 (Rust engine $\ll$ LLM on memory CWEs) originally rested on $n = 21$ Juliet C/C++ cases targeting memory-safety CWEs. While Wilson CIs are disjoint (Rust: $[0.08; 0.40]$ vs. LLM: $[0.85; 1.0]$), the small sample constitutes the most significant limitation of this work. The result has been extended to $n = 60$ (pilot). The finding is confirmed at $n = 60$ (McNemar $p < 0.001$, CIs disjoint: Rust $[0.12; 0.32]$ vs. LLM $[0.89; 0.99]$). However, external validation on addi-

tional C/C++ memory-CWE corpora remains open. The Rust engine, as implemented, shows substantially lower recall than the LLM on this sub-corpus, but a definitive conclusion requires $n \geq 60$ memory-CWE cases with diverse syntactic variants. We retain the label “H2 refuted directionally” throughout and *do not* generalize to all specialized Rust static analyzers, which may use different rule sets. This is the most important caveat of this paper.

2. *CVEfixes noisy negatives*: “clean” cases are the fixed version, which may still contain vulnerabilities. Precision/FPR/Youden are invalid on this dataset; only recall is interpreted.
3. *Partial reproducibility*: local randomness is fixed (seed=0) and model versions frozen. Hosted LLMs (Groq/OpenRouter/NVIDIA) do not guarantee bit-exact determinism: variance is attenuated by reporting CIs and magnitude conclusions.
4. *Vul4J sample selection*: the 6 excluded cases (files too large, commits without `src/main`) may bias the distribution toward simpler vulnerabilities, making the 12.3 % rate slightly optimistic.

External validity.

1. *Synthetic benchmark*: OWASP is SAST-optimized; the 0.96 recall does not transfer to production.
2. *Temporal contamination (controlled)*: recall on CVE-2024 (0.72 [0.55; 0.84], $n = 32$) is higher than pre-2022 (0.21 [0.17; 0.27], disjoint CIs), ruling out inflation from memorized old CVEs.
3. *Model coverage (T8)*: tested models are constrained by free-tier access (Groq, NVIDIA NIM, OpenRouter); results may not generalize to frontier proprietary models. In particular, GPT-4o (OpenAI) and Claude 3.5 Sonnet (Anthropic) — the current industrial references — were not evaluated due to access constraints. This is an open external-validity threat: the detection/repair trade-off ($\rho_s = -0.71$) and the 24.6 % repair ceiling may shift substantially for models with significantly larger context windows or code-specific fine-tuning. A 60-case pilot on one frontier model is designated Future Work #9.

Construct validity.

1. *H3 bounded*: the 0/183 bound relies on two static scanners insensitive to logical flaws; this is an instrumental lower bound, not a proof of safety — reported as such throughout.
2. *Similarity as proxy*: explicitly refuted as a repair metric; only PoV measurements are presented as primary results.

Conclusion validity.

1. Per-model CIs remain wide for individual fix-rates ($n = 183$); only the aggregate rate (127/1031, CI [10.4 %; 14.4 %]) is a reliable estimate. Inter-model rankings must be read with caution.
2. The detection/repair decoupling is confirmed at $\alpha = 0.05$ ($\rho_s = -0.71$, $p = 0.021$, $n = 10$ models, two-tailed t -approximation). Extension to additional

C/C++ memory-CWE corpora is required for full external validation.

7.8 Synthesis: Claims vs. Scope

Established with disjoint CIs: multi-SAST synergy (H1); H2 refutation (LLM $\gg$ Rust on memory CWEs); superiority of executable tests over similarity. **Bounded but not exhaustive**: patch safety (H3, 0 confirmed vulnerability on bi-scanner; logical flaws not covered). **Directional, to be confirmed**: generalization beyond Vul4J (detection/repair decoupling now confirmed: $\rho_s = -0.71$, $p = 0.021$, $n = 10$).

7.9 Formal Taxonomy of LLM Patch Failure Modes

We formalize the empirical failure taxonomy (Table 19) into a predictive theoretical framework. Let V be a vulnerability in file F at location L , and let T be the PoV executable test.

Definitions.

- *edit_dist(V)*: minimum number of AST node modifications to eliminate V .
- *scope(V)*: number of files whose modification is necessary and sufficient to eliminate V (formally: $|\{f \in \text{call_graph}(L) : \text{fix}(V) \cap f \neq \emptyset\}|$).
- *ctx_sufficiency(V, LLM)*: indicator that all code relevant to V fits within the LLM context window (1 if $\text{relevant_region}(V) \leq \text{ctx_limit}$).

Failure mode sufficient conditions (derived from 80-case manual inspection):

1. **Type A (Residual)**: $\text{edit_dist}(V) > 1$ and the LLM selects the minimal-surface edit. Predicted when the vulnerability root cause is semantically distant from its syntactic manifestation (e.g., CWE-502, CWE-400). *Empirical rate*: 38.8 % of failures.
2. **Type B (Scope mismatch)**: $\text{scope}(V) > 1$. The fix crosses file boundaries; single-file patching is provably insufficient. *Empirical rate*: 30.0 %.
3. **Type C (Context loss)**: $\text{ctx_sufficiency}(V, \text{LLM}) = 0$ (file > 15 k chars in our configuration). *Empirical rate*: 17.5 %.
4. **Type D (Semantic side effect)**: patch compiles but PoV fails at location $L' \neq L$ due to unintended behavioral change. Occurs when the LLM over-generalizes the fix. *Empirical rate*: 13.8 %.

Theoretical repair ceiling (informal estimate).

Equation 5 derives a conservative estimate of the achievable repair rate if the dominant structural barrier (Type B, single-file constraint) were removed. The derivation isolates Type B as the single bottleneck, treating the other failure types (A, C, D) as independent from Type B for the purpose of this calculation. *Independence caveat*: in practice, a case may satisfy both Type B and Type A simultaneously (multi-file scope and residual semantic error); the empirical overlap in the inspected sample is $\leq 12\%$ (3 of 24 Type B cases also classified Type A). Under the independence

assumption the bound is:

$$\begin{aligned} \text{PoV-rate}_{\max} &= 1 - P(\text{Type B}) \cdot (1 - r) \\ &= 1 - 0.30 \times (1 - 0.123) \approx \mathbf{0.386} \end{aligned} \quad (5) \quad (6)$$

where $r = 0.123$ is the observed repair rate. **Sensitivity analysis:** if Type B and Type A overlap by 50 % (pessimistic case), the effective removable fraction drops to $P(\text{TypeB}_{\text{pure}}) \approx 0.15$, yielding a lower bound of ≈ 0.26 . The range $[0.26, 0.39]$ brackets the plausible improvement from adding multi-file support — a $2\times$ – $3\times$ gain. This estimate is falsifiable: any system with multi-file patching should achieve $\geq 26\%$ PoV fix-rate on Vul4J under our protocol (vs. our 12.3 %).

External validation. The Text4Shell (PoV PASS) and Log4Shell (PoV PASS) production cases satisfy scope=1 (single-file fix). The Spring4Shell failure (PoV FAIL) satisfies scope=2 $\rightarrow$ Type B as predicted. This provides 3-case empirical support for the taxonomy’s predictive validity.

7.10 The MAS-SEC Evaluation Framework as a Reusable Methodological Artifact

Beyond specific measurements, this work formalizes a **reusable evaluation framework** (MAS-SEC) for security-oriented LLM agents. The framework has three properties that, taken together, distinguish it from prior evaluation approaches:

Property P1 — Executable verification primacy. All repair claims are anchored in executable PoV tests, not textual similarity. Formally: a patch p for vulnerability v is *confirmed* if and only if $\text{PoV}(v, p) = \text{PASS}$, where PoV is the executable test suite associated with v in Vul4J.

Property P2 — Causal ablation. Each system component X has a *causal contribution estimate* computed by $\Delta_X = \text{metric}(\text{full}) - \text{metric}(\text{full} \setminus X)$, where $\text{full} \setminus X$ is the pipeline with X disabled via a configuration flag, all other parameters fixed. This guarantees that Δ_X measures X ’s *isolated* contribution, not a confound.

Property P3 — Calibrated uncertainty. Every rate estimate is accompanied by a confidence interval: Wilson CI for proportions (repair rates), bootstrap CI for aggregate metrics (F1). Claims are classified as *established* (disjoint CIs), *directional* (overlapping CIs but consistent trend), or *bounded instrumental* (H3: zero-introduction on limited scanner set). This classification prevents overstatement.

This framework is *instantiable*: any practitioner can substitute a new LLM (replacing DeepSeek-V3), a new SAST tool (replacing Semgrep), or a new benchmark (replacing Vul4J) into the protocol and obtain comparable, interpretable measurements. The GitHub repository provides the configuration stubs to do so.

8. CONCLUSION AND FUTURE WORK

This empirical study establishes, on a corpus of 183 PoV-verifiable cases, six reproducible findings of direct scientific value.

Key results. (1) H1 supported with disjoint CIs: multi-SAST orchestration raises recall from 0.79 to 0.96 on synthetic OWASP. (2) H2 refuted for this specific Rust engine: the general-purpose LLM dominates on memory CWEs (recall 1.00 vs. 0.19, disjoint CIs). (3) The orchestrated pipeline outperforms LLM-only by +15.9 absolute points fix-rate PoV (24.6 % vs. 8.7 %) — empirical proof of architectural added value. (4) Detection/repair decoupling is **confirmed at** $\alpha = 0.05$ across ten LLMs ($\rho_s = -0.71$, $p = 0.021$): no model dominates both axes, directly motivating a bi-model pipeline. (5) The PoV repair rate (12.3 %, CI [10.4 %; 14.4 %], $n = 1031$) is substantially below published functional APR rates (30–47 %), quantifying the *security-verification gap*. (6) CWE-level analysis reveals a clear pattern: success on syntactically simple flaws (SQLi 55 %, XSS 60 %), failure on semantically complex ones (deserialization 25 %, XXE 29 %).

Architectural implications. These results directly guide CI/CD pipeline design: (i) multi-SAST orchestration is necessary for recall on synthetic benchmarks; (ii) the LLM is the decisive layer on real code but must be orchestrated; (iii) a bi-model pipeline (llama-3.3-70b for detection, deepseek-v3 for repair) is **suggested by the data** and warranted for further investigation; (iv) future efforts must target semantically complex CWEs via taint analysis.

H3 conclusion. Zero confirmed new vulnerabilities were introduced over 183 patches as measured by a SAST bi-scanner (syntactic lower bound), corroborated by a behavioral manual audit on $n = 8$ patches spanning three logical CWE categories (CWE-284, CWE-362, CWE-840) yielding 0 new vulnerabilities. H3 is thus extended from a purely syntactic bound to a **partially behavioral bound**. A full behavioral audit on all 183 patches remains open (future work, Future Work #3).

Main contribution. This work formalizes a reusable evaluation framework (MAS-SEC, Section 7.10) with three formal properties: executable verification primacy (P1), causal ablation (P2), and calibrated uncertainty (P3). The framework is instantiable with any LLM, SAST tool, or benchmark. It rigorously documents negative and directional results — often absent from the literature — and identifies three evaluation pitfalls: noisy negatives (CVEfixes), textual similarity as a failing proxy, and synthetic corpus bias (OWASP). The CWE-level analysis provides the first direct evidence, on a PoV-verified corpus, of a systematic performance cliff between syntactically-simple and semantically-complex vulnerabilities, with direct design implications for CI/CD integration.

Future work:

1. **H2 robustness:** extend memory-CWE evaluation to $n \geq 60$ C/C++ cases with diverse syntactic variants (buffer overflow patterns, use-after-free, integer overflow) to confirm or qualify the H2 directional refutation — the primary open question of this study.

2. **Exhaustive H3**: extend patch safety to executable security tests (not just 2 SAST scanners); paired runs with/without ValidatorAgent.
3. **Bi-model pipeline**: empirically evaluate llama-3.3-70b (detection) + deepseek-v3 (repair) in a sequential pipeline on the full 183-case Vul4J corpus.
4. **Full-scale RAG** (transferred from preliminary pilot): evaluate on all 798 CVEfixes cases; the 60-case pilot (recall $0.24 \rightarrow 0.31$, partially overlapping CIs) shows a positive trend but no quantitative claim is made until full evaluation.
5. **FPR reduction**: integrate taint/data-flow analysis downstream of semantic LLM reasoning to reduce CVEfixes FPR (currently 0.79).
6. **JavaScript**: complete ESLint Security Plugin + Snyk Code integration; validate 10-case pilot gains (recall $0.10 \rightarrow 0.30$) on full 148-case JS corpus.
7. **SecurityEval/CyberSecEval extension**: adapt PoV verification to SecurityEval’s 75 CWE scenarios and CyberSecEval’s insecure-code-detection dimension for cross-benchmark comparison.
8. **Corpus expansion**: extend Vul4J to $n \geq 400$ with holdout CVE 2024–2025 for tighter CIs and generalization testing.
9. **Production code evaluation**: all current benchmarks (OWASP, Juliet, CVEfixes, Vul4J) use curated or synthetic corpora. A case study on a real production repository (candidate: an open-source Java application with ≥ 10 known CVEs) is required to assess generalization beyond benchmark distributions and to detect context-dependent vulnerabilities absent from synthetic datasets.

ACKNOWLEDGMENTS

The authors thank ENSAM Meknès and the faculty of the AI & Data Technology program, NVIDIA for NIM access (70B models), Groq for free-tier access (LLaMA 3.1 8B), and OpenRouter for access to additional models.

References

- [1] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” *ICLR*, 2024.
- [2] R. Just, D. Jalali, M. D. Ernst, “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs,” *ISSTA*, 2014.
- [3] OWASP Foundation, “OWASP Benchmark Project v1.2,” *Technical Documentation*, 2024.
- [4] NIST SAMATE, “Juliet Test Suite for C/C++ and Java,” *SAMATE Registry*, 2023.
- [5] G. Bhandari, A. Naseer, L. Moonen, “CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software,” *PROMISE*, 2021.
- [6] Q.-C. Bui, R. Scandariato, N. E. Díaz Ferreyra, “Vul4J: A Dataset of Reproducible Java Vulnerabilities Geared Towards the Study of Program Repair Techniques,” *MSR*, 2022.
- [7] C. Le Goues, T. Nguyen, S. Forrest, W. Weimer, “GenProg: A Generic Method for Automatic Software Repair,” *IEEE TSE / ICSE*, 2012.
- [8] F. Long, M. Rinard, “Automatic Patch Generation by Learning Correct Code,” *POPL*, 2016.
- [9] M. Monperrus, “Automatic Software Repair: A Bibliography,” *ACM Computing Surveys*, 51(1), 2018.
- [10] K. Liu, A. Koyuncu, D. Kim, T. F. Bissyandé, “TBar: Revisiting Template-based Automated Program Repair,” *ISSTA*, 2019.
- [11] J. Xuan *et al.*, “Nopol: Automatic Repair of Conditional Statement Bugs in Java Programs,” *IEEE TSE*, 2017.
- [12] X. B. D. Le, D. Lo, C. Le Goues, “History Driven Program Repair,” *SANER*, 2016.
- [13] H. Pearce, B. Tan, B. Ahmad, R. Karri, “Examining Zero-Shot Vulnerability Repair with Large Language Models,” *IEEE S&P*, 2023.
- [14] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, D. Phung, “VulRepair: A T5-based Automated Software Vulnerability Repair,” *ESEC/FSE*, 2022.
- [15] B. Ahmad, B. Tan, H. Pearce, R. Karri, “No More Copy-Paste? A Large-Scale Study of Vulnerable Code Clones,” *IEEE S&P Workshops*, 2021.
- [16] H. Li *et al.*, “Large Language Models for Automatic Vulnerability Repair,” *ICSE NIER*, 2024.
- [17] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, “Devign: Effective Vulnerability Identification via Graph Neural Networks,” *NeurIPS*, 2019.
- [18] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, “Deep Learning Based Vulnerability Detection: Are We There Yet?,” *IEEE TSE*, 48(9), 2022.
- [19] M. Fu, C. Tantithamthavorn, “LineVul: A Transformer-based Line-Level Vulnerability Prediction,” *MSR*, 2022.
- [20] D. Huang *et al.*, “AgentCoder: Multi-Agent-based Code Generation with Iterative Testing,” *arXiv:2312.13010*, 2024.
- [21] J. Yang, C. E. Jimenez *et al.*, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” *NeurIPS*, 2024.
- [22] H. Ghaleb *et al.*, “A Systematic Literature Review of Automated Program Repair Techniques,” *TOSEM*, 2024.
- [23] J. Liu *et al.*, “Deep Learning for Vulnerability Detection: A Systematic Literature Review,” *IEEE TSE*, 2023.
- [24] G. Mariano *et al.*, “Automated Program Repair in the Era of Large Language Models,” *arXiv:2405.18765*, 2024.
- [25] Cognition AI Team, “Introducing Devin, the First

- AI Software Engineer,” *Technical Report*, 2024.
- [26] GitHub, “GitHub Copilot Workspace,” *Technical Documentation*, 2024.
- [27] Semgrep Inc., “Semgrep: Lightweight Static Analysis for Many Languages,” *Technical Documentation*, 2024.
- [28] SpotBugs Project, “SpotBugs with FindSecBugs Plugin,” *Technical Documentation*, 2024.
- [29] Meta AI, “The Llama 3 Herd of Models,” *Technical Report*, 2024.
- [30] DeepSeek AI, “DeepSeek-V3 Technical Report,” *arXiv:2412.19437*, 2024.
- [31] LangChain Inc., “LangGraph: Orchestrating Stateful Multi-Agent Applications,” *Technical Documentation*, 2024.
- [32] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv:2107.03374*, 2021.
- [33] M. L. Siddiq, J. C. Santos, “SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques,” *MSR4PS @ ESEC/FSE*, 2022.
- [34] Meta FAIR, “CyberSecEval 2: A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models,” *arXiv:2404.13161*, 2024.
- [35] B. Rozière *et al.*, “Code Llama: Open Foundation Models for Code,” *arXiv:2308.12950*, 2023.
- [36] A. Lozhkov *et al.*, “StarCoder 2 and The Stack v2: The Next Generation,” *arXiv:2402.19173*, 2024.

MAS Research Unit Report — ENSAM 2026.

[https:](https://github.com/hinimdoumorsia/MultiAgentSecurite)

[//github.com/hinimdoumorsia/MultiAgentSecurite](https://github.com/hinimdoumorsia/MultiAgentSecurite)